ЛАБОРАТОРНА РОБОТА №2

NoSQL у системі спортивних подій

PostgreSQL, MongoDB, Redis і .NET 8

Виконавець	Кашинський Владислав, група ТТП-32
Предмет	Сучасні технології баз даних
Репозиторій	sports-events-nosql-lab
Рік	2026

Перевірка репозиторію, реалізації та benchmark-результатів

1. Мета роботи

Показати практичне використання різних моделей NoSQL у системі спортивних подій. PostgreSQL зберігає нормалізовані дані та транзакційні зв'язки, MongoDB формує денормалізовану read-модель каталогу, а Redis використовується для кешу, лічильників і рейтингу популярності.

2. Перевірений склад репозиторію

• postgres/schema.sql, seed.sql, benchmark-seed.sql і SQL-еквівалент NoSQL-моделі.

• mongodb/init.js зі схемою валідації та трьома індексами.

• mongodb/queries.js з find, text search, aggregation та explain.

• redis/queries.txt з командами key-value, hash, set і sorted set.

• .NET 8 застосунок у src/SportsEvents.NoSqlLab.

• docker-compose.yml для PostgreSQL, MongoDB і Redis.

• benchmark/results.json з відтворюваним результатом вимірювання.

• README.md і REPORT.md.

Основна гілка містить один коміт 4cf004a4a374326aa980a0facca214783a801154 від 07.11.2026.

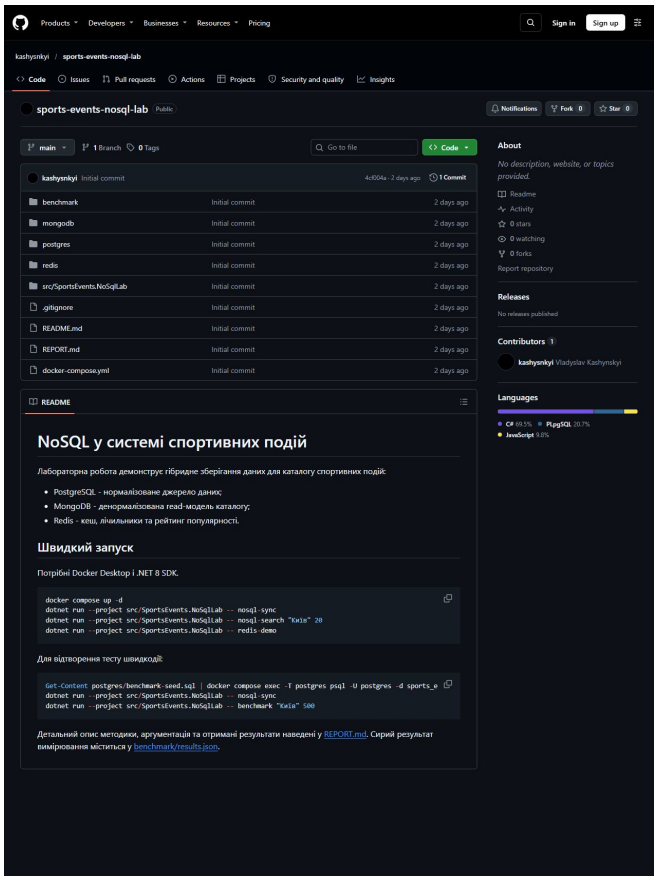


Рисунок 1 - Сторінка репозиторію GitHub і структура лабораторної

3. Архітектура рішення

Компонент	Роль
PostgreSQL	Джерело істини, зв'язки, обмеження, soft delete, реєстрації та SQL-представлення.
MongoDB	Колекція event_catalog з вкладеним спортом, локацією, організатором і категоріями.
Redis	Кеш картки події з TTL, hash лічильників, set міст і sorted set популярності.

Компонент	Роль
.NET 8	Синхронізація PostgreSQL -> MongoDB, пошук, benchmark і демонстрація Redis.
Docker Compose	Єдиний запуск інфраструктури з трьома сервісами.

4. MongoDB read-модель

• event_catalog містить вкладені sport, venue, organizer і categories.

• confirmedRegistrations зберігається як попередньо агреговане поле.

• ReplaceFromPostgresAsync читає SQL-view, виконує bulk upsert і видаляє застарілі документи.

• Створено compound B-tree, text index і descending index за lastModifiedAt.

5. Redis

• SET/GET картки події з TTL 10 хвилин.

• HSET/HGETALL лічильників переглядів і заявок.

• SADD/SMEMBERS множини подій міста.

• ZADD/ZREVRANGE рейтингу популярності.

6. Еквівалент у PostgreSQL

postgres/nosql-equivalent.sql створює view v_event_catalog_sql і функцію fn_search_event_catalog. View збирає картку події через шість нормалізованих таблиць і формує категорії як JSONB-масив. PostgreSQL збирає картку під час запиту, MongoDB читає готовий документ.

7. Benchmark

benchmark-seed.sql створює 10 000 подій, 30 000 категорій і 360 000 підтверджених реєстрацій. Використано 8 прогрівальних і 500 основних ітерацій. У кожній ітерації перевіряється кількість і множина id результатів.

Сховище	Mean	Median	P95	Min	Max
PostgreSQL, мс	33.831	39.041	50.440	14.111	60.747
MongoDB, мс	2.109	1.588	4.188	0.804	40.275

Обидва сховища повертали однакові 50 id подій. У цьому read-heavy сценарії MongoDB мала середню затримку у 16.04 раза меншу. Це не означає загальної переваги MongoDB: PostgreSQL залишається джерелом істини для транзакцій, унікальності та зв'язків.

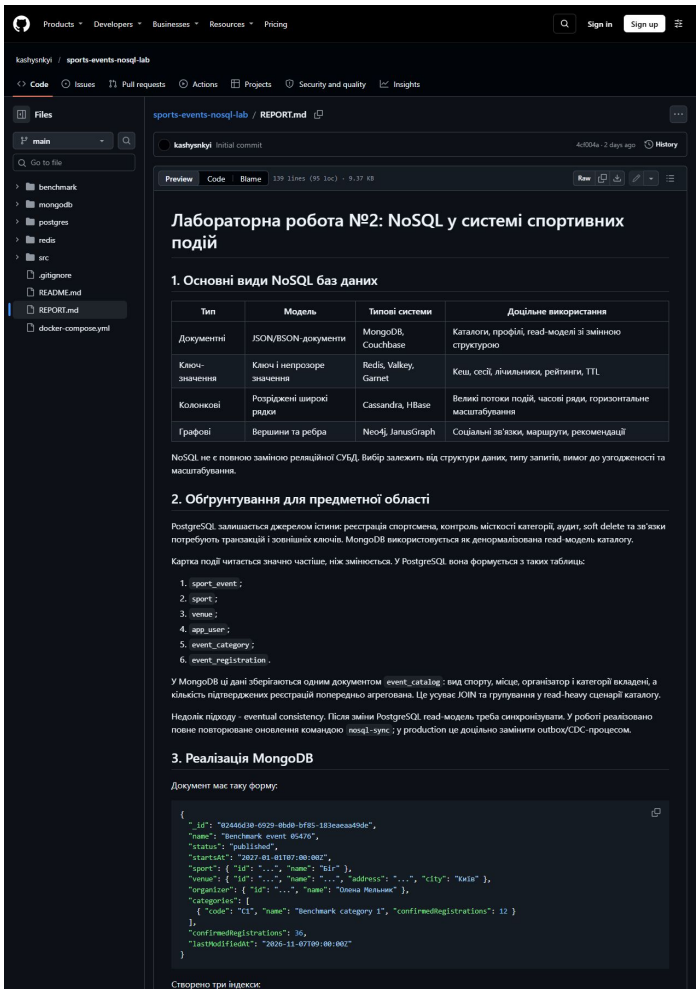


Рисунок 2 - REPORT.md з описом методики та benchmark-результатів

8. Перевірка працездатності

• Репозиторій клоновано з GitHub, структура файлів перевірена.

• Проєкт .NET 8 зібрано у Release: 0 помилок і 0 попереджень.

• Команду довідки запущено успішно: nosql-sync, nosql-search, benchmark, redis-demo.

• benchmark/results.json узгоджений: 500 ітерацій, 50 результатів з кожної БД, speedup 16.044.

• У текстових файлах не виявлено небажаних згадок про генератори або сторонні інструменти.

Повний інтеграційний запуск Docker у цьому середовищі не виконувався, оскільки Docker CLI недоступний. SQL, MongoDB init-скрипт, Compose і .NET-код перевірені статично, а компіляція .NET пройшла успішно.

9. Висновок

Друга лабораторна реалізована як гібридна NoSQL-система. PostgreSQL зберігає узгоджені нормалізовані дані, MongoDB прискорює читання каталогу через денормалізовану модель, а Redis обслуговує кеш та рейтинги. Репозиторій містить код, конфігурацію Docker, SQL-скрипти, запити, benchmark і документацію.